

CloudGuard: Explainable Policy Violation Detection and Remediation Guidance for AWS Infrastructure as Code

Uday and Project Team

Abstract. Infrastructure as Code (IaC) makes cloud deployment repeatable, but insecure configuration can be propagated at the same speed. This paper presents CloudGuard, a browser-based prototype for local analysis of AWS Terraform and CloudFormation files. CloudGuard detects five policy conditions: public S3 access, publicly accessible RDS, unrestricted security-group ingress, wildcard IAM actions, and disabled RDS encryption. Each finding includes a policy identifier, severity, risk deduction, control reference, and remediation guidance. The implementation is intentionally scoped to guidance and transparent scoring; it does not claim automatic repair, Kubernetes or Helm support, expert annotation, or human-subject evidence. A reproducible project-team synthetic mutation benchmark contains 50 manifests and 70 labelled policy-instance pairs. On this benchmark, the prototype produced 30 true positives, 0 false positives, 0 false negatives, and 40 true negatives, yielding precision, recall, and F1 of 1.0000. These values are reported only as an implementation sanity check because the benchmark was constructed around the same five rules. The paper defines an evidence-first protocol for future baseline comparison, ranking, remediation validation, and usability studies.

Keywords: Infrastructure as Code, cloud security, policy violation detection, explainable risk, Terraform, CloudFormation.

1. Introduction

Cloud infrastructure is increasingly declared in machine-readable configuration rather than provisioned manually. This improves repeatability, but it also allows a permissive access rule or disabled protection to be copied across environments before deployment review. Security scanners can identify many conditions, yet their output is often difficult for a developer or reviewer to connect to a concrete policy and next action.

CloudGuard addresses this traceability problem with a small, inspectable AWS IaC analyzer. The prototype reads Terraform and CloudFormation files locally in the browser, applies five explicit rules, computes a transparent score from configured deductions, and displays remediation guidance. It is a capstone research artifact and demonstration, not a production cloud security service.

Minna et al. [1] investigated inconsistent security-policy analysis and policy mapping across analyzers for Helm charts in Kubernetes environments. CloudGuard studies the related traceability problem for AWS Terraform and CloudFormation, without implementing Helm or Kubernetes support.

The contributions are: (1) a browser workflow for local AWS IaC policy review; (2) a finding representation that preserves policy ID, severity, risk points, source-control reference, and guidance; (3) a reproducible synthetic mutation benchmark for implementation sanity testing; and (4) an explicit boundary between measured results and future experiments.

2. Scope and Research Questions

CloudGuard supports Terraform and CloudFormation only. It does not support Helm or Kubernetes, does not execute changes against AWS, and does not automatically rewrite source files. The selected file is read in the browser, matched against deterministic rules, and rendered as a report and remediation view.

RQ1 asks whether the rule set identifies the policy mutations represented in a frozen synthetic AWS IaC benchmark. RQ2 asks whether findings can be presented with explainable severity, risk points, and remediation guidance. RQ3 asks whether controlled remediation guidance removes the targeted policy violations without introducing additional structural errors in the synthetic benchmark.

3. System Design

The user selects a .tf, .yaml, .yml, or .json file, or loads the built-in vulnerable sample. The browser places the text in the editor. Matching rules become finding records. The renderer displays a report, a 0-100 score, and remediation recommendations. No cloud credentials or deployment operation is required.

The current policies are S3-001 for public S3 access, RDS-001 for public RDS access, SG-001 for unrestricted ingress, IAM-001 for wildcard IAM actions, and RDS-002 for disabled database encryption. The score begins at 100 and subtracts 28, 25, 18, 18, and 14 points for these findings respectively. The score is an attention-prioritization aid, not a probability of compromise.

Policy ID	Condition	Severity	Points
S3-001	Public S3 access	Critical	28
RDS-001	Public RDS access	Critical	25
SG-001	Unrestricted ingress	High	18
IAM-001	Wildcard IAM actions	High	18
RDS-002	Disabled encryption	High	14

Table 1. Implemented CloudGuard policies and deductions.

4. Benchmark and Reproducibility

The research package contains a 50-manifest project-team synthetic mutation benchmark: 25 Terraform and 25 CloudFormation manifests, with 70 labelled policy-instance pairs. A policy instance associates a file, resource, format, policy SID, expected violation label, and severity. This avoids treating one file as one result when several policies occur in the same file.

The labels were produced by the project team using predefined criteria. Two reviewer files are retained as an audit trail, but the collection process is not documented as independent blinded annotation. Therefore, this paper does not report Cohen's kappa or claim inter-rater reliability. The benchmark is suitable for regression testing and implementation sanity checks, but not for generalization to arbitrary production configurations.

Results are regenerated by the repository reproduction script from retained labels and predictions. The public repository is <https://github.com/duday8329-ai/cloudguard-research>; its availability and contents should be checked again at submission time.

5. Evaluation Method

For CloudGuard, each policy-instance prediction is compared with the expected violation label. Precision is $TP/(TP+FP)$, recall is $TP/(TP+FN)$, and F1 is $2PR/(P+R)$. Baseline-analyzer metrics are reserved for future comparison after validated semantic mappings are established. Unmapped findings are not silently treated as negatives.

Latency is measured over repeated local analyzer runs and reported with the environment and trial count. These values describe the prototype on the synthetic benchmark, not a production service. Any future remediation study must record each case, generated action, validation result, and failure category. Any human study must use consent and appropriate oversight; participant records must never be invented.

Metric	Measured value	Interpretation
Manifests	50	25 Terraform + 25 CloudFormation synthetic mutations
Policy instances	70	Project-team labels at policy-instance level
TP / FP / FN / TN	30 / 0 / 0 / 40	Generated CloudGuard sanity matrix
Precision / Recall / F1	1.0000 / 1.0000 / 1.0000	Optimistic synthetic result; not generalization evidence
Latency	10 trials; median 66.091 ms; p95 84.382 ms	Local Windows prototype measurement

Table 2. Reproducible results currently available in the package.

6. Results

The CloudGuard sanity run generated 30 true positives, no false positives, no false negatives, and 40 true negatives over 70 policy-instance labels. Precision, recall, and F1 are all 1.0000. Because the mutation cases were designed to exercise the same five prototype rules, this result is expected to be optimistic. It demonstrates that the pipeline, labels, predictions, and metric generation work together; it does not demonstrate perfect performance on unseen AWS IaC.

A local latency run used 10 trials. The median was 66.091 ms and the p95 was 84.382 ms on the recorded Windows environment. The measurement should not be compared directly with hosted services or tools running under different conditions.

Raw baseline scans for Checkov, KICS, and Trivy are retained in the package, but comparative precision, recall, and F1 are defensible only for policy SIDs with validated semantic mappings. This paper therefore makes no broad superiority claim. A future comparison must freeze the mapping, store raw outputs, normalize findings to the same policy-instance unit, and regenerate the tables.

7. Controlled Remediation Guidance Validation

Thirty positive policy instances were copied into before/after cases. For each target case, a manually prepared remediation was applied to the corresponding configuration, after which CloudGuard re-scanned the modified configuration: public access was restricted, encryption was enabled, unrestricted CIDRs were narrowed, and wildcard actions were replaced with a specific action. All 30 cases passed the recorded structural check and all 30 target violations disappeared on re-scan. Native Terraform and CloudFormation validators were unavailable, so Gate 1 is a structural integrity check. The result measures controlled remediation guidance clearance and does not establish automatic repair capability.

8. Leave-One-Policy-Out Ablation

The full system produced precision 1.0000, recall 1.0000, and F1 1.0000. Removing any one of the five policy rules produced precision 1.0000, recall 0.8000, and F1 0.8889, with six false negatives. Each policy family contributes six positive cases in this balanced synthetic benchmark. These results show sensitivity to removal of individual rules within the synthetic benchmark, not general model robustness.

9. Ranking Consistency Check

The CloudGuard-ranked findings produced NDCG@10 of 1.0000. The relevance labels were assigned with the predefined severity and risk-point rubric: public S3 and public RDS received 4, unrestricted ingress and wildcard IAM received 3, and disabled encryption received 2. This is a consistency check against the same rubric used by the ranking, not evidence of independent ranking quality or expert-consensus NDCG.

10. Limitations and Evidence Boundary

The current package does not contain a human-participant study, production configurations, automatic AST-based repair, native IaC validation logs, or a large external benchmark. Checkov, KICS, and Trivy outputs are retained, but broad comparative claims require validated semantic mappings for every compared policy instance. The implementation is pattern-based and can miss equivalent configurations expressed through variables, modules, intrinsic functions, indirection, or uncommon syntax.

It does not build a full Terraform or CloudFormation semantic model. Remediation output is advice rather than an AST transformation, and the score is a transparent prioritization aid rather than a calibrated risk model.

11. Future Evaluation Protocol

A larger study should add manually reviewed Terraform and CloudFormation examples while preserving policy-instance ground truth. The same files should be scanned by CloudGuard, Checkov, KICS, and Trivy using a published normalized mapping. Ranking relevance should be assigned before any system ranking is shown. Remediation cases should use native Terraform and CloudFormation validators. Any developer study requires consent and appropriate oversight.

12. Conclusion

CloudGuard is a compact, explainable AWS IaC analyzer ready for demonstration and reproducible synthetic evaluation. Its current strengths are transparency, stable policy IDs, visible deductions, local file analysis, generated metrics, controlled remediation guidance checks, and component-sensitivity results. The benchmark supports an honest prototype evaluation, not a broad production-accuracy claim. The ranking value is a consistency check against the predefined rubric, not independent ranking-quality evidence. A conference submission should preserve this evidence boundary, use the venue's official template, and attach the repository and raw artifacts.

References

- [1] F. Minna, A. Blaise, K. Tuma, and F. Massacci, "Automated Analysis of Security Policy Violations in Helm Charts," IEEE Transactions on Dependable and Secure Computing, vol. 23, no. 2, pp. 2519-2533, 2026, doi: 10.1109/TDSC.2025.3628213.
- [2] AWS, AWS Well-Architected Framework, Security Pillar, <https://docs.aws.amazon.com/wellarchitected/latest/security-pillar/welcome.html>
- [3] Bridgecrew, Checkov documentation, <https://www.checkov.io/>
- [4] Checkmarx, KICS documentation, <https://docs.kics.io/>
- [5] Aqua Security, Trivy documentation, <https://trivy.dev/>
- [6] HashiCorp, Terraform language documentation, <https://developer.hashicorp.com/terraform/language>
- [7] AWS, CloudFormation User Guide, <https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/Welcome.html>